

SCP/SCPC Command Interface — Build Notes, Wiring, Firmware, and Operation

Note from the author

This repo was summarized by LLM based on lab notes and may contain errors I haven't caught. Feel free to contact me with any issues.

License

Copyright © 2026 Matthew Stubler.

The Arduino firmware is licensed under the PolyForm Noncommercial License 1.0.0. The original documentation, diagrams, and photos are licensed under Creative Commons Attribution-NonCommercial 4.0 International. Commercial use requires separate permission from the author.

See the license files in the repository for the complete terms.

Research use and disclaimer

Research prototype — not for clinical or patient-care use.

- Use only with perfusion equipment permanently removed from clinical service. Do not return modified or interfaced equipment to clinical service.
- Developed and demonstrated on a surrogate-fluid benchtop circuit; not validated as a medical device or for human use.
- Animal, cadaveric, isolated-organ, or other preclinical applications were not validated by this project. Any such use requires independent technical assessment, protocol-specific validation, and applicable institutional, regulatory, and ethical approval.
- Hardware, software, and documentation are provided as is, without warranty.
- Anyone building, modifying, connecting, or operating this project does so at their own risk and assumes responsibility for the resulting equipment behavior, safety, and application.

The author assumes no responsibility for injury, equipment damage, regulatory consequences, or other loss resulting from use or misuse of this project.



What this is

A removable Arduino-based interface for the decommissioned Sorin SCP/SCPC centrifugal pump.

The interface sits at the ZPR 9909 A / CON2 connection and leaves the native pump electronics, motor controller, CLOCK, FRAME, TACH, and local reference in place. Only the DATA signal is intercepted.

- Relay off: native DATA passes directly to the motor-control board.
- Relay on, FRAME high: buffered native DATA passes through the interface.
- Relay on, FRAME low: Arduino-generated DATA replaces native DATA.
- Native CLOCK and FRAME provide communication timing; the Arduino does not generate them.
- Deenergizing the relay returns DATA control to the native panel. It does not stop the pump.

Firmware source is provided separately as `scp_scpc_pressure_control.ino`.

The demonstrated firmware can:

- decode the native 16-bit command field (**CMD16**);
- reproduce the native command;
- apply bounded command offsets;
- adjust the command in response to an analog pressure signal;
- return to native DATA control when the programmed control bound is reached.

Hardware required

Core interface

- Permanently decommissioned matching Sorin SCP/SCPC pump system.
- Arduino Uno R4 Minima and USB data cable.
- Half-size Perma-Proto board.
- SN74HC125N tri-state buffer.
- P2N2222/P2N2222A transistor; verify the actual device pinout before assembly.
- DFRobot DFR0017 relay module.
- Two 1-k Ω resistors.
- Two 10-k Ω resistors.
- 0.1- μ F decoupling capacitor.
- Regulated external 5-V supply.
- Hookup wire, insulation, strain relief, soldering supplies.
- Multimeter and compatible logic analyzer.

Removable Y-harness

Fabricate a removable Y-tap harness from DuPont-style connectors and leads.

- A standard 2 × 3 female DuPont connector housing mates with ZPR 9909 A / CON2.
- Preserve straight-through TACH, CLOCK, FRAME, and GND.

- Split native DATA so it can still be monitored upstream of the relay.
- Route the interface-selected DATA output back to the motor-control board.

Verify connector orientation and every conductor before applying power.

Pressure-control demonstration

Additionally requires a compatible 5v analog pressure transducer (a generic 0-30psi one from aliexpress was used for the demo), independent pressure measurement, centrifugal pump head and tubing, reservoir and surrogate test fluid, and an adjustable downstream restriction.

How it connects

The interface installs at ZPR 9909 A / CON2 using the removable Y-harness.

Signal	Interface handling
TACH	Straight-through; optional monitoring only
CLOCK	Straight-through; tapped for Arduino timing
FRAME	Straight-through; tapped for Arduino timing and DATA selection
GND	Common local reference
DATA	Intercepted and routed through the relay/interface

DATA path

- Relay off: native DATA → motor-control board.
- Relay on, FRAME high: buffered native DATA → motor-control board.
- Relay on, FRAME low: Arduino replay DATA → motor-control board.
- Native CLOCK and FRAME continue to define communication timing.
- The Arduino does not generate CLOCK or FRAME.
- Relay fallback restores native DATA control; it does not stop the pump.

Arduino connections

Function	Arduino
CLOCK	D2
FRAME	D3
Native DATA monitor	D4
TACH, optional	D5
DEBUG, optional	D6
Replay DATA	D7
Relay control	D8
Pressure sensor	A0

Basic use

1. Install the interface and removable Y-harness with the pump powered off.
2. Complete the pre-power checks before connecting the interface to the pump.
3. Power the interface and Arduino; open the Serial Monitor at 115200 baud.
4. Start the pump under normal native control.
5. Allow the firmware to acquire and validate the native command stream before attempting to engage the relay.
6. Select manual command-offset mode or pressure-control mode as needed.

Serial commands

Input	Function
P or p + Enter	Start pressure-control mode
r + Enter	Start manual command-offset mode
Signed integer + Enter	Request that CMD16 offset from the captured baseline in manual mode
Blank Enter	Return to native control; also clears a latched control-limit error
n + Enter	Same as blank Enter

Manual command-offset mode

Enter **r** to engage manual mode. The firmware captures the current accepted native CMD16 as the baseline and initially replays that value without an offset.

Enter a signed integer to request an offset from the captured baseline.

Examples:

- **10** = baseline + 10 CMD16 counts
- **-10** = baseline - 10 CMD16 counts
- **0** = return the synthetic target to the captured baseline

Requested offsets are limited to ± 65 CMD16 counts.

In the tested system, 1 CMD16 count corresponded roughly to 3 RPM of displayed pump-speed change. This relationship varied with hydraulic load and was not calibrated; treat it only as a rough estimate, not a direct RPM conversion or measurement.

The ± 65 -count working limit therefore corresponds roughly to ± 195 RPM under similar conditions, but should not be treated as a fixed RPM limit.

Pressure-control limit

If pressure control requests a correction reaching the ± 65 -count bound:

- the firmware latches a control error;
- synthetic control is disabled;
- the relay is deenergized;
- DATA control returns to the native pump panel.

The pump is not stopped by this action.

Clearing the error does not restart pressure control. Correct the test condition as needed, then deliberately start pressure-control mode again to capture a new target and baseline.

Return to native control

Press Enter on a blank line or enter `n`.

This deenergizes the relay, disables manual and pressure-control modes, restores the direct native DATA path, and clears a latched control-limit error if present.

Returning to native control does not stop the pump. Pump operation remains under the native control panel.

Relay-engagement checks

The firmware does not energize the relay immediately at startup. Before relay engagement it requires:

- at least 25 processed frames;
- at least 10 accepted command frames;
- an available accepted replay frame;
- 16-bit current-native and stored-replay commands;
- agreement within 0x0020 CMD16 counts between the latest decoded native command and stored replay command.

If these conditions are not met, relay engagement is blocked and native DATA remains selected.

Command acceptance limits

Accepted CMD16 range:

- Minimum: 0x0000
- Maximum: 0x0700

While relay control is selected, a newly decoded command differing from the previously accepted command by more than 0x0200 counts is rejected rather than becoming the new replay value.

These are firmware acceptance limits, not calibrated pump-performance limits.

CON2 connector reference

Use DATA, CLOCK, FRAME, TACH and GND on pump-facing labels. Official CON2 contact numbers appear only in the chart below; Perma-Proto rows, component pins and Arduino pins retain their own numbering.

CON2 contact (manual)	Wire	Group	Signal label	Observed role
1	Yellow	3-wire	DATA	Bit state
2	Green	3-wire	CLOCK	DATA timing
3	Green	2-wire	FRAME	Active low
4	Yellow	2-wire	TACH	feedback/RPM-associated proxy
5	Black (sheath)	3-wire	GND	Local reference

Connector numbering, wire colors and cable groups follow the official service-manual convention; signal assignments follow the supplied JECT working-article mapping. TACH denotes the feedback/RPM-associated proxy, interpreted as tachometer feedback; reliable Arduino pulse acquisition was not established, and this signal is observation-only, not used for control.

Board Rules

- Board type: half-size Perma-Proto / breadboard-style layout.
- 30 rows.
- A-E = left half.
- F-J = right half.
- A-E are connected together per row.
- F-J are connected together per row.
- A-E are not connected to F-J unless a jumper crosses the center gap.
- Side rails carry +5V and GND only.
- HC125 straddles the center gap.
- HC125 notch faces toward row 01.
- 0 = dead / unused area.
- - = row is active on that half, but this specific slot is unused.

Side Rails

- Left + rail = +5V
- Left - rail = GND
- Right + rail = +5V
- Right - rail = GND

- Bridge left/right +5V rails only if needed.
 - Bridge left/right GND rails only if needed.
 - Meter rails before applying power.
-

Board Layout Table

ROW	A	B	C	D	E		F	G	H	I	J
01	TACH	LA-T	TAP-T	TACH	TACH		0	0	0	0	0
02	DATA	LA-D	ARD-D4	TAP-D	DATA		0	0	0	0	0
03	CLOCK	ARD-D2	LA-C	TAP-C	CLOCK		0	0	0	0	0
04	FRAME	ARD-D3	LA-F	TAP-F	FRAME		0	0	0	0	0
05	POUT	LA-PO	TAP-PO	POUT	POUT		0	0	0	0	0
06	0	0	0	0	0		0	0	0	0	0
07	0	0	0	0	0		RLY-D	RLY-D	RLYCTL	-	-
08	NC	NC	DATA	TAP-NC	-		0	0	0	0	0
09	NO	NO	MUX	TAP-NO	-		0	0	0	0	0
10	COM	COM	POUT	TAP-COM	-		0	0	0	0	0
11	0	0	0	0	0		ARD-D8	RLYCTL	LA-RLY	TAP-RLY	-
12	0	0	0	0	0		ARD-D7	REPLAY	LA-RP	TAP-RP	-
13	0	0	0	0	0		ARD-D6	DEBUG	LA-DBG	TAP-DBG	-
14	0	0	0	0	0		0	0	0	0	0
15	-	-	-	TAP-OE1	/OE1		+5V	TAP-5V	-	-	-
16	-	-	-	TAP-A1	A1		/OE4	TAP-OE4	-	-	-
17	-	-	R1A	TAP-Y1	Y1		A4	TAP-A4	-	-	-
18	-	-	-	TAP-OE2	/OE2		Y4	-	-	-	-
19	-	-	-	TAP-A2	A2		/OE3	TAP-OE3	-	-	-
20	-	-	R2A	TAP-Y2	Y2		A3	TAP-A3	-	-	-
21	-	-	-	TAP-GND	GND		Y3	-	-	-	-
22	0	0	0	0	0		0	0	0	0	0
23	MUX	LA-MUX	R1B	R2B	MUX		0	0	0	0	0
24	INV	LA-INV	TAP-INV	INV	INV		0	0	0	0	0
25	0	0	0	0	0		0	0	0	0	0
26	Q-E	Q-E	-	-	-		0	0	0	0	0
27	Q-B	RB1	-	-	-		0	0	0	0	0
28	Q-C	RP1	INV	INV	INV		0	0	0	0	0
29	0	0	0	0	0		0	0	0	0	0
30	0	0	0	0	0		0	0	0	0	0

Legend

- TACH = pump feedback/RPM-associated proxy.
 - DATA = native pump DATA.
 - CLOCK = pump CLOCK.
 - FRAME = pump FRAME.
 - POUT = switched DATA return back to pump.
 - MUX = DATA_MUX node after HC125 outputs pass through 1k resistors.
 - INV = inverted FRAME from P2N2222 collector.
 - NC = relay normally-closed contact.
 - NO = relay normally-open contact.
 - COM = relay common contact.
 - RLY-D = relay module D / IN pin.
 - RLYCTL = Arduino D8 relay-control signal.
 - REPLAY = Arduino D7 replay / synthetic DATA output.
 - DEBUG = Arduino D6 optional debug output.
 - LA-* = logic analyzer tap.
 - TAP-* = extra tap / labeled jumper termination point.
 - R1 = 1k resistor from replay HC125 output Y1 to MUX.
 - R2 = 1k resistor from native HC125 output Y2 to MUX.
 - RB1 = 10k base resistor from FRAME to transistor base.
 - RP1 = 10k pullup resistor from transistor collector to +5V.
 - Q-E = P2N2222 emitter.
 - Q-B = P2N2222 base.
 - Q-C = P2N2222 collector.
-

Connector Plan and Wiring Harness

Harness placement

- Bench use only with permanently decommissioned equipment. No patient connection. Animal or other preclinical use requires independent validation and applicable approval, as stated above.
- Access point: ZPR 9909 A / CON2 beneath the removable drive-unit cover.
- Panel side = existing cable/filter path from ZKR 9910 / CON2. Motor-board side = ZPR 9909 A / CON2.
- Fit the removable harness between the original cable connector and the motor-board connector. Leave native power, motor-winding and Hall-sensor connections unchanged.
- Label pump-facing connections DATA, CLOCK, FRAME, TACH and GND; distinguish panel-side DATA from motor-board DATA.

Straight-through signals and taps

- TACH, CLOCK, FRAME and GND: maintain their original panel-to-motor-board connections.
- Parallel branches: TACH → row 01; CLOCK → row 03; FRAME → row 04; local GND → GND rail.
- CLOCK and FRAME provide native timing; the Arduino observes them, not drives them.

1. Pump Input Connector

- Place at top-left of the interface board.

- TACH branch → row 01 TACH.
- Panel-side native DATA → row 02 DATA.
- CLOCK branch → row 03 CLOCK.
- FRAME branch → row 04 FRAME.
- Local GND → GND rail.

2. Pump Output Connector

- Motor-board DATA ← row 05 POUT.
- Local GND → GND rail.
- CLOCK/FRAME may pass through the board connector via rows 03/04, or remain straight through in the harness. Their native connections must remain intact.

DATA routing

- Only DATA is interrupted in substitution mode. Do not leave a parallel native-DATA jumper bypassing the relay.
- Native monitoring: row 02 DATA → Arduino D4, upstream of the relay; monitoring continues in both relay states.
- Deenergized: row 02 DATA → row 08 NC → relay COM → row 10 POUT → row 05 POUT → motor-board DATA.
- Energized: row 23 MUX → row 09 NO → relay COM → row 10 POUT → row 05 POUT → motor-board DATA.
- MUX selects buffered native DATA with FRAME high and Arduino D7 DATA with FRAME low. See HC125 Wiring and P2N2222 Inverter Wiring.

Connect and check

- Remove pump and external-interface power before installing, removing or changing the harness; account for USB power to the Arduino.
- Confirm connector orientation before mating; use the CON2 reference chart to identify signals. Check continuity; do not rely on wire color alone.
- Secure and insulate the removable connections; avoid loose contacts and exposed conductors.
- Join pump local GND, interface GND, external-supply GND and Arduino GND. Follow the existing Power rule below.
- Before pump connection, check connector orientation and continuity; verify that NC connects native DATA to motor-board DATA with relay power off; check for a parallel DATA bypass; and verify the FRAME-dependent routing with the pump unpowered.
- Keep the bypass relay installed. Start with it deenergized and confirm native control before enabling substitution; do not reset or upload to the Arduino while the pump is powered.
- Relay fallback restores native DATA; it does not stop the pump.
- To restore the original wiring: remove all power, remove the interface harness, and reconnect the original cable directly to ZPR 9909 A / CON2.

Arduino Connections

- Arduino GND → GND rail.
- Arduino D2 → row 03 ARD-D2 / CLOCK.
- Arduino D3 → row 04 ARD-D3 / FRAME.
- Arduino D4 → row 02 ARD-D4 / DATA.
- Arduino D7 → row 12 ARD-D7 / REPLAY.
- Arduino D8 → row 11 ARD-D8 / RLYCTL.

Optional:

- Arduino D5 → row 01 TACH.
- Arduino D6 → row 13 DEBUG.

Power rule:

- External 5V supply powers +5V rail.
 - External supply GND powers GND rail.
 - Arduino GND connects to GND rail.
 - Arduino 5V stays disconnected unless intentionally powering the board from Arduino.
-

Relay Module Connections

1. Relay Logic Side

- Relay D / IN → row 07 RLY-D.
- Relay + → +5V rail.
- Relay - → GND rail.

Required jumper:

- row 11 RLYCTL → row 07 RLY-D.

2. Relay Contact Side

- Relay NC → row 08 NC.
- Relay NO → row 09 NO.
- Relay COM → row 10 COM.

Required relay contact jumpers:

- row 08 DATA → row 02 DATA.
- row 09 MUX → row 23 MUX.
- row 10 POUT → row 05 POUT.

Relay behavior:

- Relay OFF: DATA → NC → COM → POUT → pump.
- Relay ON: MUX → NO → COM → POUT → pump.

This preserves native DATA fallback when relay is not energized.

HC125 Placement

Place the 74HC125 across the center gap.

- HC125 notch faces row 01.
- Left pins land on E15–E21.
- Right pins land on F15–F21.

HC125 pin map:

- E15 = pin 1 /OE1.

- E16 = pin 2 A1.
 - E17 = pin 3 Y1.
 - E18 = pin 4 /OE2.
 - E19 = pin 5 A2.
 - E20 = pin 6 Y2.
 - E21 = pin 7 GND.
 - F21 = pin 8 Y3.
 - F20 = pin 9 A3.
 - F19 = pin 10 /OE3.
 - F18 = pin 11 Y4.
 - F17 = pin 12 A4.
 - F16 = pin 13 /OE4.
 - F15 = pin 14 +5V.
-

HC125 Wiring

1. HC125 Power

- F15 +5V → +5V rail.
- E21 GND → GND rail.

Add decoupling capacitor close to chip:

- 0.1 μ F capacitor from F15 +5V to E21 GND.

2. Replay Channel

- E16 A1 → row 12 REPLAY.
- E15 /OE1 → row 04 FRAME.
- E17 Y1 → R1A.
- R1A → 1k resistor → R1B.
- R1B → row 23 MUX.

3. Native Channel

- E19 A2 → row 02 DATA.
- E18 /OE2 → row 24 INV.
- E20 Y2 → R2A.
- R2A → 1k resistor → R2B.
- R2B → row 23 MUX.

4. Unused HC125 Channels

- F16 /OE4 → +5V rail.
 - F17 A4 → GND rail.
 - F19 /OE3 → +5V rail.
 - F20 A3 → GND rail.
 - F18 Y4 → open.
 - F21 Y3 → open.
-

P2N2222 Inverter Wiring

Place transistor in rows 26–28 left.

Use the actual transistor pinout. For the previous working orientation:

- Flat face toward you.
- Emitter → Q-E.
- Base → Q-B.
- Collector → Q-C.

Connections:

- row 26 Q-E → GND rail.
- row 04 FRAME → 10k resistor → row 27 RB1 / Q-B.
- row 28 Q-C → row 24 INV → E18 / OE2.
- row 28 Q-C → 10k pullup resistor → +5V rail.

Hard rule:

- Collector never connects directly to +5V.
 - Collector connects to +5V only through the 10k pullup.
-

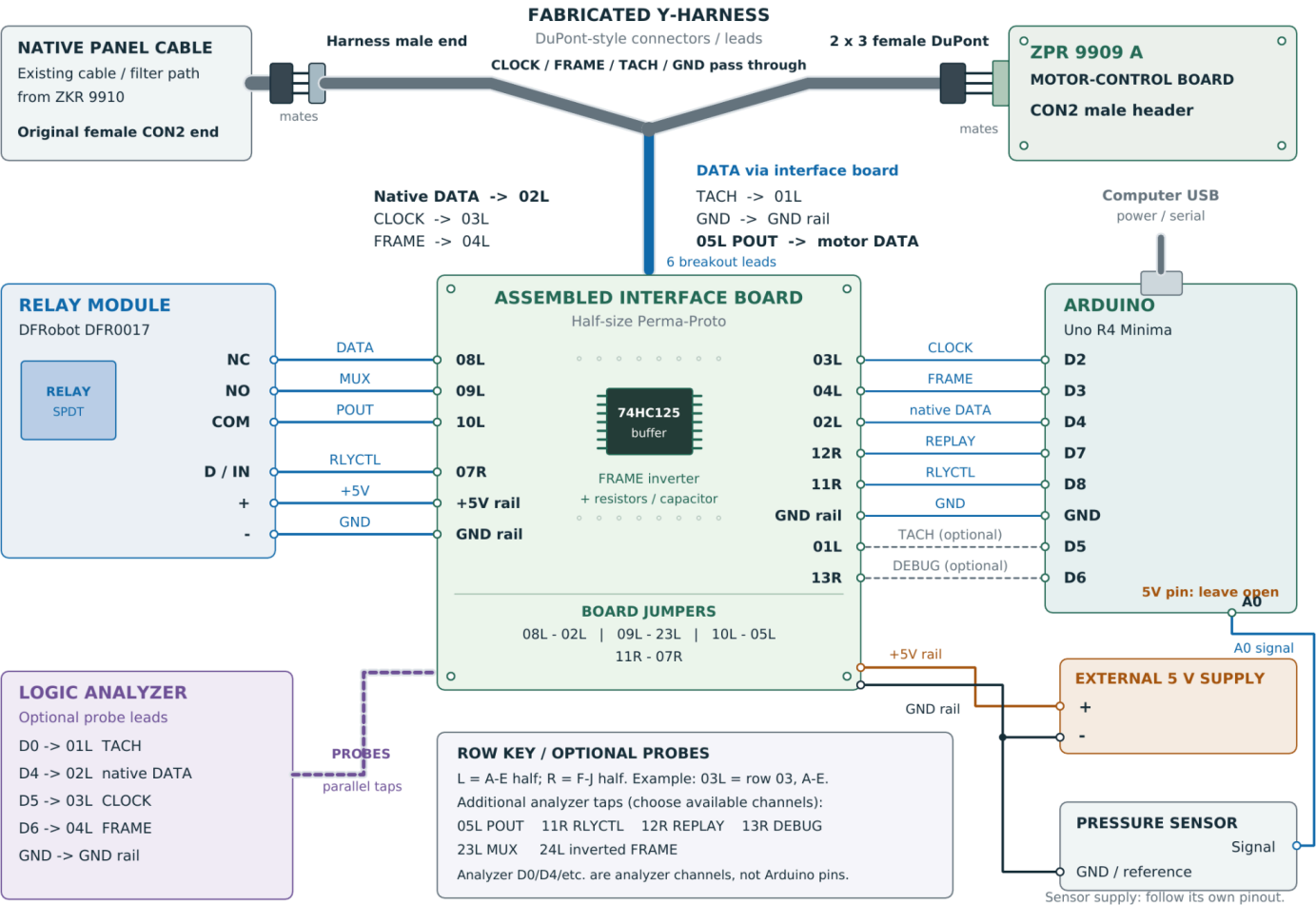
Logic Analyzer Taps

- GND → GND rail.
- LA-T → row 01 TACH.
- LA-D → row 02 DATA.
- LA-C → row 03 CLOCK.
- LA-F → row 04 FRAME.
- LA-PO → row 05 POUT.
- LA-RLY → row 11 RLYCTL.
- LA-RP → row 12 REPLAY.
- LA-DBG → row 13 DEBUG.
- LA-MUX → row 23 MUX.
- LA-INV → row 24 INV.

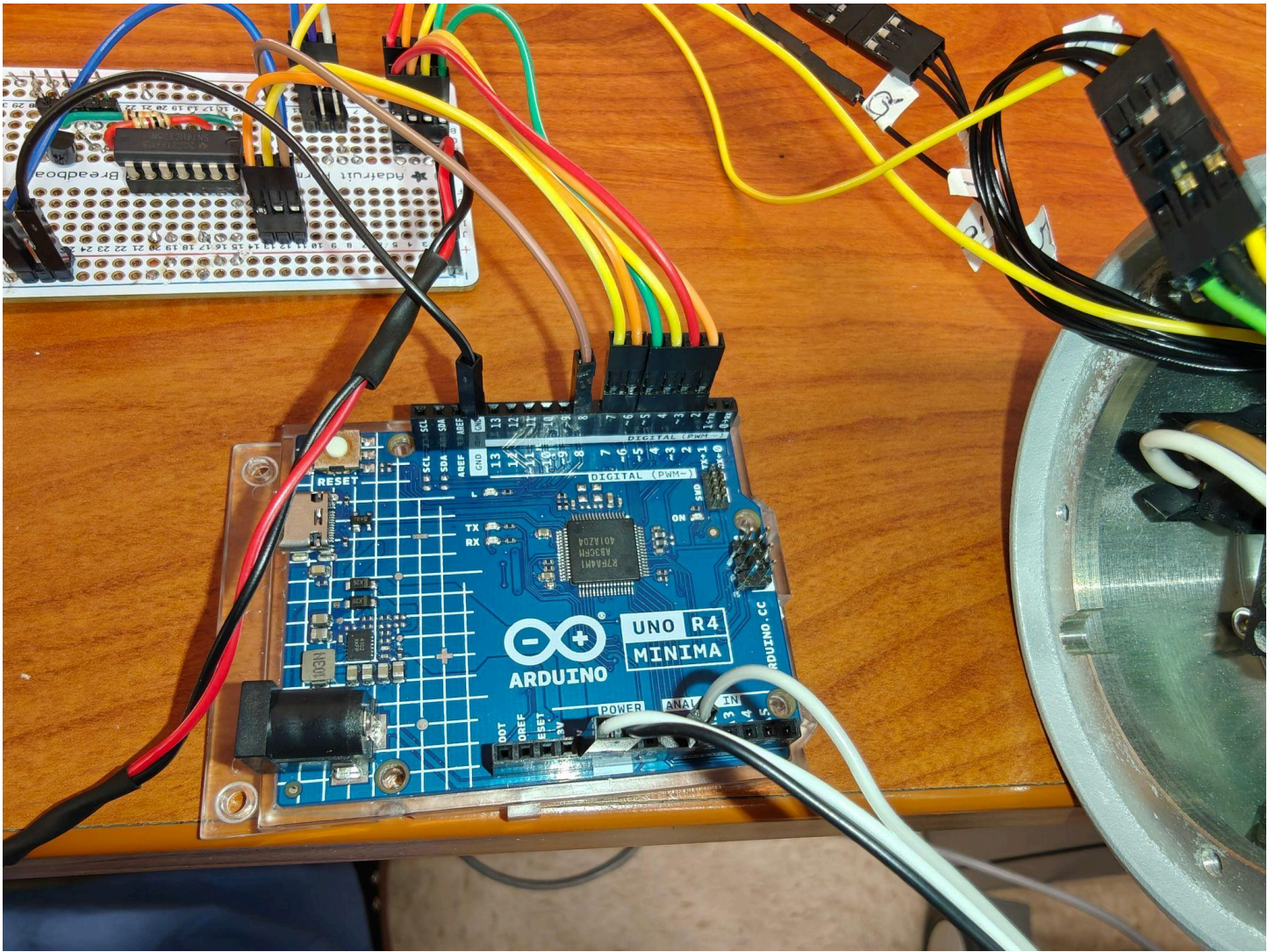
SCP / SCPC | What connects to what

ASSEMBLY WIRING

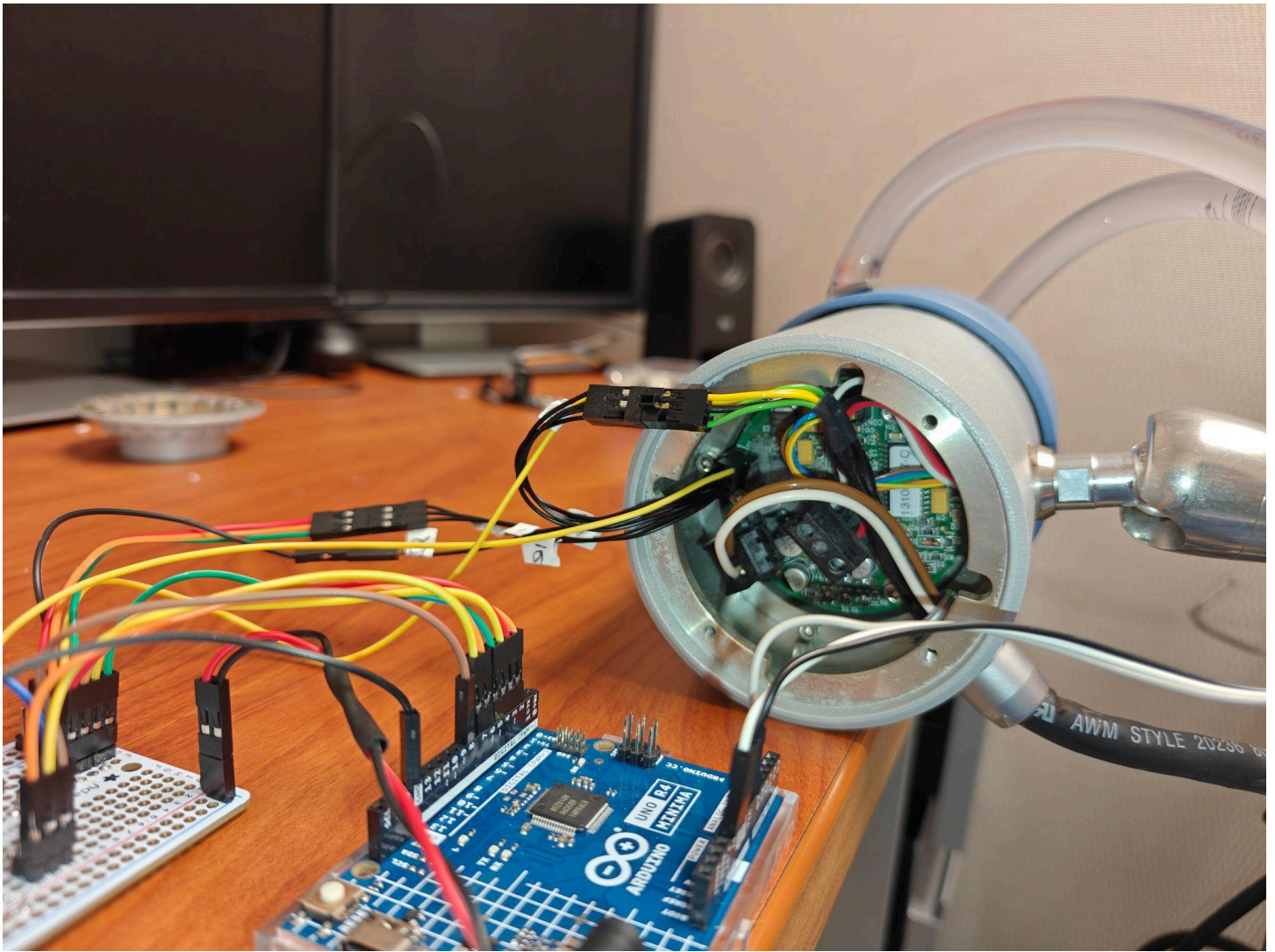
Removable Y-harness, assembled interface board, relay, Arduino and optional analyzer



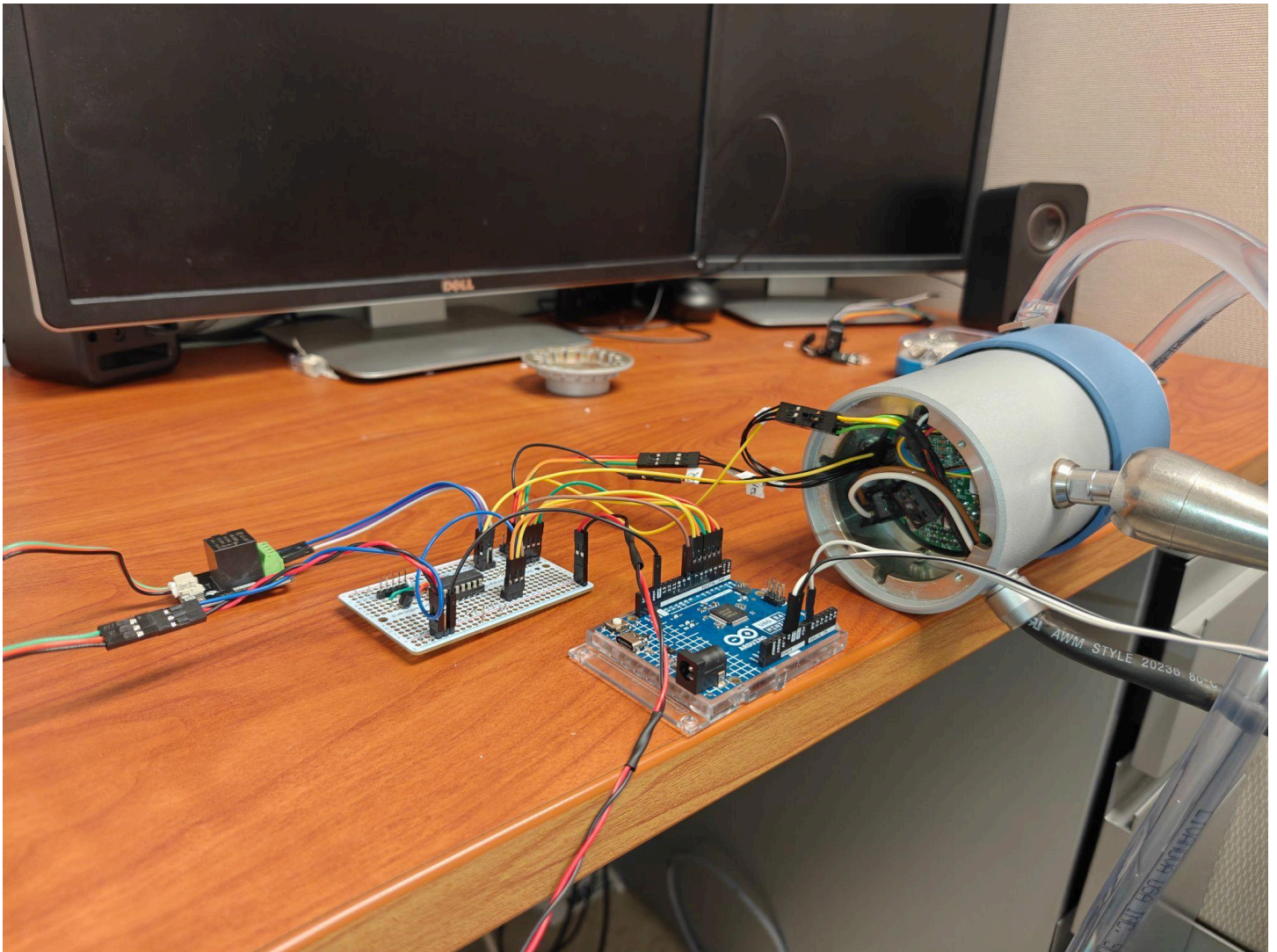
Connector views / port positions are symbolic. Verify CON2 orientation by signal; obsolete logical pin numbers are omitted.



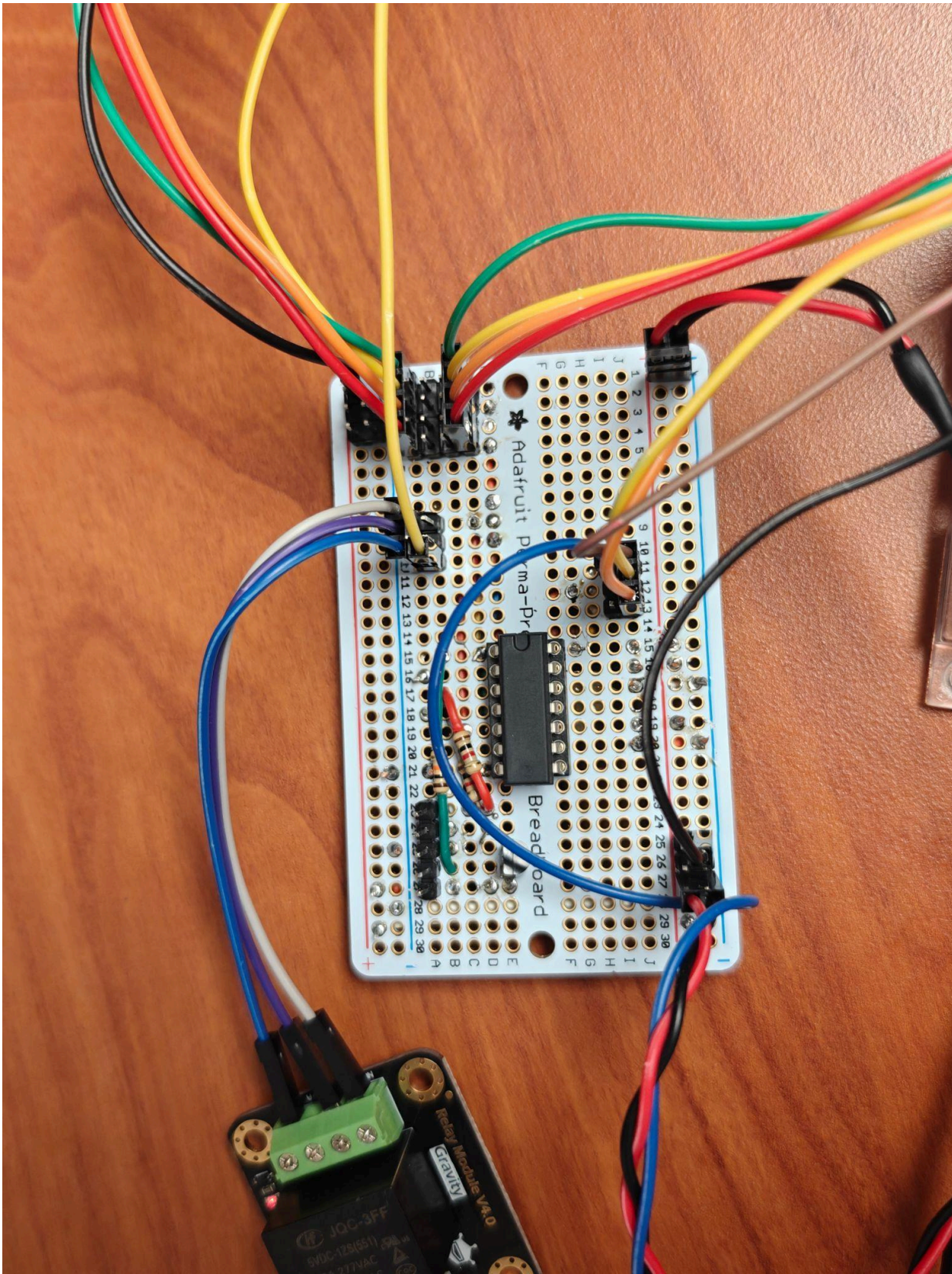
Arduino wiring and harness leads.



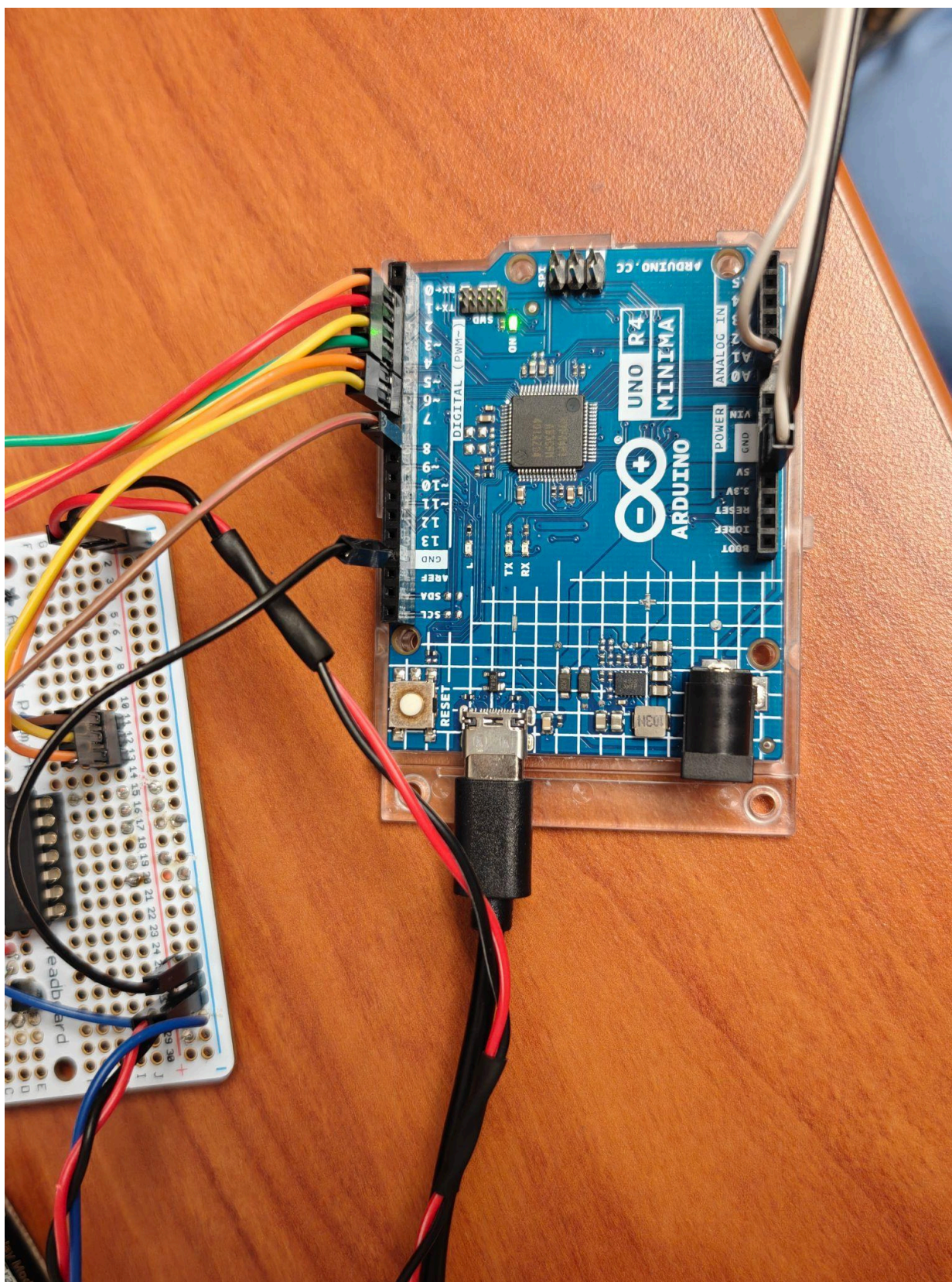
CON2 connection at the opened drive unit.



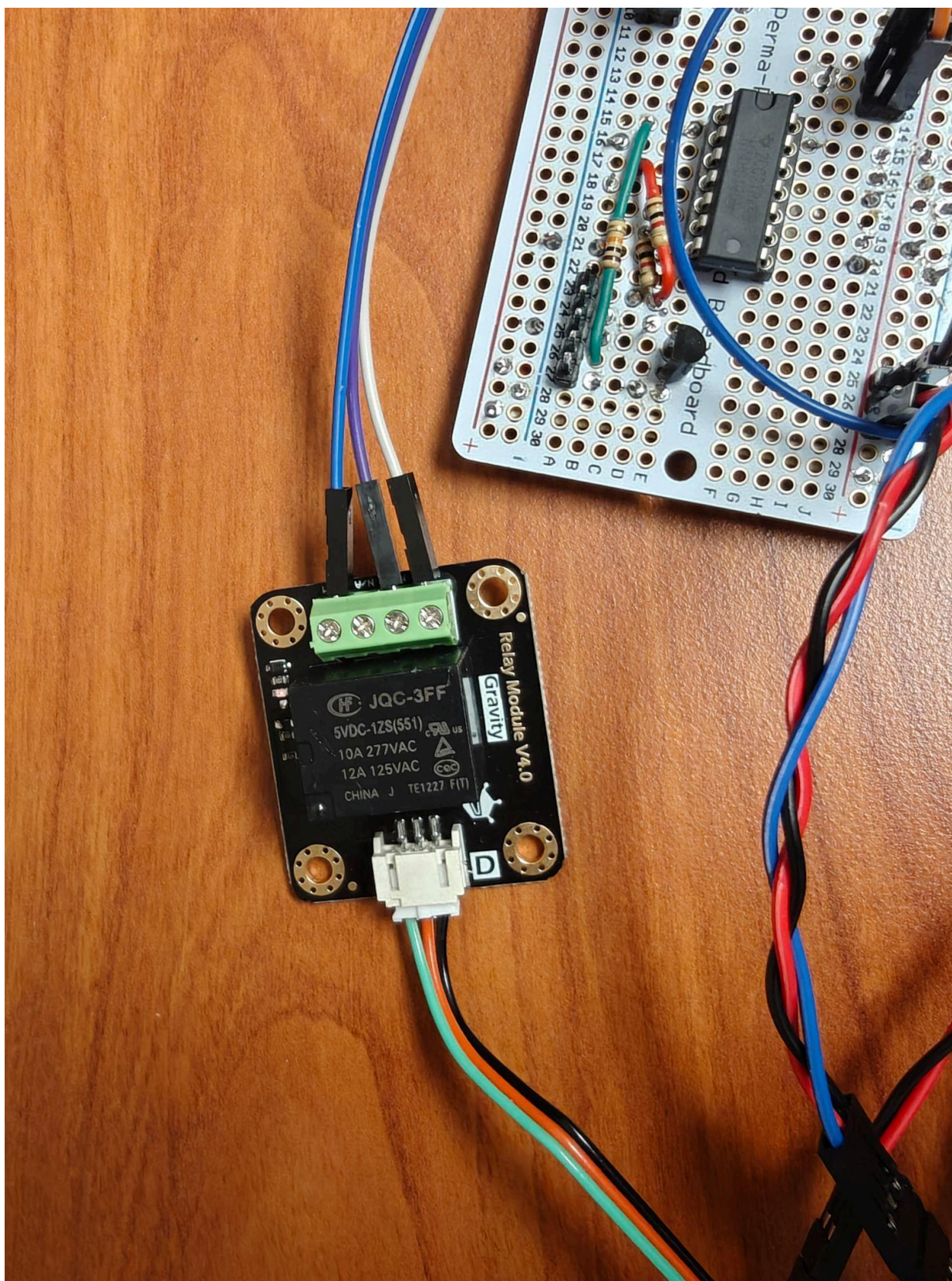
Bench setup: harness, interface board, Arduino, and pump.



Interface board and relay, top view.



Arduino connections, top view.



Relay module and interface board, close-up.